

CS360 Introduction to Database

Final Exam

(Total 100 points)

1. [8 points] 1GB of data consisting of 4KB block size is stored sequentially or randomly in HDD and SSD. We attempt to read the data using double buffering techniques and process SQL operations or Deep Learning operations. The processing time for one block in the buffer is 0.02ms in SQL and 1ms in Deep Learning. If the performance of HDD and SSD is as below, fill in the ①~⑧ with elapsed time in millisecond. (1GB = 1,000MB = 1,000,000KB)

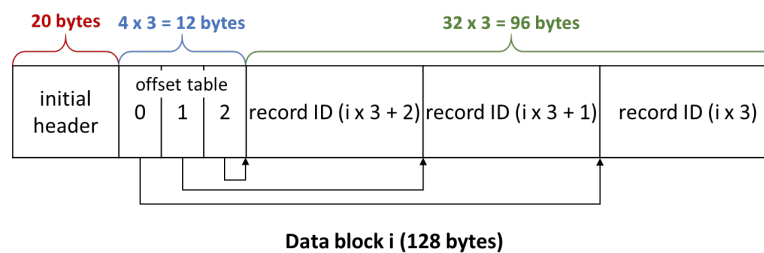
	Seek time	(Rotational) latency	Transfer time
HDD	5ms	4ms	100MB/s
SSD	0ms	0.1ms	400MB/s

		SQL	Deep Learning
Store sequentially	HDD	① ms	⑤ ms
	SSD	② ms	⑥ ms
Store randomly	HDD	③ ms	⑦ ms
	SSD	④ ms	⑧ ms

2. [15 points] There is the **Employee** relation created using SQL as shown below.

```
CREATE TABLE Employee (
    ID INT,
    name CHAR(20),
    age INT
);
```

Each record has 4 byte header, and so is 32 bytes long. In the Employee relation, a total of 105 records (ID: 0 ~ 104) are stored in the slotted page format below. One block size is 128 bytes and stores up to 3 records. As shown below, record IDs 0 ~ 2 are stored in **block 0**, record IDs 3 ~ 5 are stored in **block 1**, and record IDs $(i \times 3) \sim (i \times 3 + 2)$ are stored in **block i**.



The database address of the **block i** is denoted as **<data block i, 0>** and the database address of the **record j** within the **block i** is denoted as **<data block i, offset j>** ($0 \leq j < 3$).

The B-tree index is constructed on the age attribute. The block size of B-tree index is 128 bytes, where the size of a key is 4 bytes, and the size of a pointer in database address is 12 bytes. An index block consists of 7 keys and 8 pointers. Therefore, the B-tree index on Employee.age consists of 1 root node, 2 interior nodes, and 15 leaf nodes. Suppose that the index blocks are numbered in the BFS order, starting with the root. So, the database address of the root node is **<index block 0, 0>**, the database address of the first interior node is **<index block 1, 0>**, the database block of the first leaf node is **<index block 3, 0>**. Suppose that the **age** attribute value and **ID** attribute value have the relationship of **age = ID / 2 + 10**.

During processing the below SQL query by using the B-tree index, fill the translation table entries. We consider the swizzling on demand mode. The root block of the B-tree index is already loaded in main memory at 0x0000. The second block of either B-tree index or data file will be loaded at 0x0080 (because the size of a block is 128 bytes). Unnecessary blocks for the query are not loaded in main memory. The entries for the necessary index blocks would be inserted into the translation table, and then, the entries for the necessary data blocks and their records would be inserted into the translation table. The memory address of an unswizzed pointer in the translation table should be marked as '-'.

```
SELECT name
FROM Employee
WHERE 10 <= age AND age <= 11;
```

3. [15 points] After a B-tree node is loaded into main memory, searching for a key within the B-tree node is done by in-memory binary search. The costs of in-memory and disk operations are as below. The B-tree nodes at different levels are not consecutively stored in disk.

Operation	Cost
Traversal using in-memory binary search for N keys within a B-tree node (after loading the B-tree node from disk into memory buffer)	$\lceil \log_2 N \rceil$
Loading a B-tree node from disk into memory buffer	● 100 for the first block of a B-tree node ● 0.1 for the remaining blocks of a B-tree node (when the B-tree node consists of multiple blocks)

We assume that there are 10,000,000 keys, the sizes of key and pointer are all 8 bytes, and the size of a disk block is 4KB. We consider three different sizes for a B-tree node: 4KB, 64KB, and 1MB, where 64KB and 1MB nodes consists of multiple blocks. Calculate the cost of a single-key lookup operation for the below three cases and then answer which size of a B-tree node can achieve the best lookup performance? We assume that the root node is always in memory buffer, and each B-tree node is fully occupied with keys and pointers (i.e., fill factor: 100%).

Case 1: Cost of lookup operation using the B-tree index of 4KB node:

Case 2: Cost of lookup operation using the B-tree index of 64KB node:

Case 3: Cost of lookup operation using the B-tree index of 1MB node:

What is the size of a B-tree node that achieves the best lookup performance?

4. [16 points] We assume that the two attributes of the MovieExec relation, cert# and netWorth, form a 2-dimensional data space as below. We assume that a single index block can store at most 2 index entries.

cert #	netWorth
015	\$200
021	\$150
026	\$150
034	\$200
043	\$130
048	\$150
056	\$100
062	\$150
077	\$130
089	\$100
095	\$100
100	\$130

- ① [3 points] Draw a 3-level kd-tree where the root node is of the netWorth attribute, and the size of the left child of the root is the same with the size of the right child of the root.
- ② [3 points] We assume that a single R-tree node can store up to six MBRs. So, after building a R-tree index for the above data, the R-tree consists of one root node and two leaf nodes. We assume that we have built the R-tree index such that the MBRs in the root node do not overlap with each other. List all the MBRs in the root node.
- ③ [5 points] Fill in the below table for the bitmap index using run-length encoding.

Value	compressed bit-vector

- ④ [5 points] We assume that the 10-th record, i.e., (89, 100) is deleted, and a new record of (71, 150) is inserted. Fill in the below table for the bitmap index using run-length encoding.

value	compressed bit-vector

5. [18 points] Consider a natural join between two relations $R(A, B)$ and $S(B, C)$. R and S are clustered, $B(R) = 1000$, and $B(S) = 200$. Each block of R and S holds 20 tuples. There are 51 memory buffers available. Calculate the I/O cost for each of the following join algorithms. Ignore the I/O cost of writing the final output, as you learned in class.

- ① [3 points] *Iteration Join Algorithm, R First*. The algorithm reads 50 R blocks into memory, reads all of S (1 block at a time) into memory, joining with R . The process is repeated until all outputs are generated.

Number of IOs:

- ② [3 points] *Iteration Join Algorithm, S First*. The algorithm reads 50 S blocks into memory, read all of R (1 block at a time) into memory, joining with S . The process is repeated until all outputs are generated.

Number of IOs:

- ③ [3 points] *Merge Join, Sorted Relations*. Assume that R and S are each already sorted.

Number of IOs:

- ④ [3 points] *Merge Join, Unsorted Relations*. Assume R and S are unsorted. First, R is sorted using 2PMMS, and then, its result is stored in a single sorted file. S is sorted and stored in a similar fashion. Finally, the merge-join is performed.

Number of IOs:

- ⑤ [3 points] *Index Join*. There is a NON-clustering index on the B attribute of R . We read a block of S and for each tuple s of S in the block, we find all matching tuples $\{r_1, \dots, r_m\}$ of R by using the index. The tuples $\{r_1, \dots, r_m\}$ are read into memory to be joined with s . We repeat the process for all S blocks. Assume that the index is entirely in memory buffers. Furthermore, assume that $V(R, B) = 5000$.

Number of IOs:

- ⑥ [3 points] *Hash Join*. We first partition (i.e., bucketize) R (using the B attribute) into 50 buckets (No R buckets are kept in memory.) We then partition S into 50 buckets in a similar fashion. Then we hash-join on each pair of R_i and S_i buckets.

Number of IOs:

6. [10 points] Suppose three relations, $X(a,b)$, $Y(b,c)$ and $Z(c,a)$, having the following statistics.

$X(a,b)$	$Y(b,c)$	$Z(c,a)$
$T(X) = 2000$	$T(Y) = 3000$	$T(Z) = 5000$
$V(X,a) = 50$		$V(Z,a) = 100$
$V(X,b) = 30$	$V(Y,b) = 40$	
	$V(Y,c) = 200$	$V(Z,c) = 20$

- ① [3 points] Estimate the size of $\mathbf{W} = \mathbf{X} \bowtie \mathbf{Y} \bowtie \mathbf{Z}$, i.e., $T(W)$, by using the above statistics.

- ② [3 points] Estimate the size of $\sigma_{a=10 \text{ OR } c=20}(\mathbf{Z})$ by using the above statistics.

- ③ [4 points] Estimate the size of the theta join $\mathbf{Y} \bowtie_{Y.c=Z.a} \mathbf{Z}$ by using the below equal-width histograms.

range	$Y.c$
1-10	120
11-20	160
21-30	220
31-40	140
41-50	50
51-60	150
61-70	200
71-80	60
81-90	250
91-100	80
101-200	1570

range	$Z.a$
1-10	330
11-20	540
21-30	120
31-40	650
41-50	840
51-60	460
61-70	300
71-80	680
81-90	350
91-100	730

7. [18 points] Suppose four relations, R, S, T, and U having the following statistics.

R(a,b)	S(b,c)	T(c,d)	U(d,a)
V(R,a) = 400			V(U,a) = 200
V(R,b) = 500	V(S,b) = 200		
	V(S,c) = 100	V(T,c) = 300	
		V(T,d) = 200	V(U,d) = 50

	{R}	{S}	{T}	{U}
size	2000	3000	1500	1200
cost	0	0	0	0
best plan	R	S	T	U

Fill all the empty cells in the below table according to the dynamic programming method. We only consider left-deep join trees. (Use '⋈' for the join symbol)

	{R, S}	{R, T}	{R, U}	{S, T}	{S, U}	{T, U}
size						
cost						
best plan						

	{R, S, T}	{R, S, U}	{R, T, U}	{S, T, U}
size				
cost				
best plan				